# PROJECT 6
# CVA6 RISC-V E-Trace & System Trace Integration & Profiling in Eclipse IDE

By Iqra Jawad Ahmed

# 1. Problem Statement

Integrate both the E-Trace and System Trace decoders (developed in Tasks 1 and 2) directly into the Eclipse IDE. This feature should allow users to acquire encoded trace data, decode it in real-time, and generate profiling reports (e.g., function execution time, cache misses) by correlating the decoded trace with the corresponding firmware's ELF firmware.

# 2. Learning Outcomes

- Advanced IDE plugin development and integration techniques (e.g., Eclipse RCP/CDT extensions).
- Deep understanding of software profiling methodologies using hardware trace data.
- Skill in data visualization and report generation within a professional development environment.

# 3. System Architecture Overview

The complete system is organized into three major layers: the Simulation Layer (CVA6 + Spike + Verilator), the Trace Processing Layer (E-Trace pipeline + System Trace pipeline), and the Profiling & Reporting Layer (Python analysis scripts + Eclipse Plugin). All three layers are orchestrated by the master pipeline script.

## 3.1 Architecture Layers

| Layer | Description |
|---|---|
| **Layer 1: Simulation** | CVA6 Verilator model + Spike ISA simulator. Compiles firmware ELF, runs simulation, produces trace_hart_0.dasm, trace_rvfi_hart_00.dasm, spike_raw.log |
| **Layer 2A: E-Trace Pipeline** | post-iss tool → encoder_model.py → decoder_model.py. Produces compressed N-Trace packets and verifies decode matches original PC trace. |
| **Layer 2B: System Trace Pipeline** | gen_system_trace2.py merges hart + RVFI dasm files into system_trace.txt with cycle, PC, mnemonic, event, function, register delta, and memory access columns. |
| **Layer 3A: Profiling Scripts** | func_exec_time.py computes self-cycles, CPI, and memory intensity per function. correlate_misses.py maps RTL cache misses to functions. finalize_profiling.py builds a unified report. |
| **Layer 3B: Eclipse Plugin** | Reads all CSV/TXT/HTML outputs, provides Eclipse Views for execution flow, profiling tables, and hotspot visualization. Runs the pipeline on demand. |

## 3.2 Data Flow

- fibonacci.c → (riscv64-unknown-elf-gcc) → simulation.elf
- simulation.elf → (Spike) → spike_raw.log → spike_pc_trace.txt
- simulation.elf → (Verilator) → trace_hart_0.dasm + trace_rvfi_hart_00.dasm
- spike_pc_trace.txt → (post-iss + encoder) → te_inst_raw packets → (decoder) → decoded trace
- trace_hart_0.dasm + trace_rvfi_hart_00.dasm → (gen_system_trace2.py) → system_trace.txt
- system_trace.txt → (func_exec_time.py) → function_exec_time.csv + .txt
- icache_misses.log + dcache_misses.log → (correlate_misses.py) → rtl_profiling_report.csv
- All CSVs + ELF → (finalize_profiling.py) → unified_profiling_report.{csv,txt}

# 4. System Setup & Installation

This section documents every installation and configuration step required to reproduce the complete CVA6 trace profiling environment. All commands below were run on Ubuntu 22.04 (WSL2) and are reproduced exactly as executed.

## 4.1 CVA6 Repository Clone

Clone the main CVA6 RISC-V processor repository along with all required submodules (verification environment, vendor IPs, and reference models):

```
  $ bash
git clone https://github.com/openhwgroup/cva6.git
cd cva6
git submodule update --init --recursive
```

This downloads approximately 2–3 GB of source code, including the Ariane core, RISC-V verification infrastructure (core-v-verif), and all vendor submodules.

## 4.2 System Package Installation

Install all required build tools, libraries, and development headers. These are needed to compile the RISC-V toolchain, Spike simulator, and Verilator:

```
  $ bash
sudo apt-get install autoconf automake autotools-dev curl git \
    libmpc-dev libmpfr-dev libgmp-dev gawk build-essential \
    bison flex texinfo gperf libtool bc zlib1g-dev


sudo apt install gcc git autoconf automake texinfo libtool make pkg-config
-y
```

Key packages: autoconf/automake (build system generators), libmpc/libmpfr/libgmp (mathematical precision libraries needed by GCC), bison/flex (parser generators), texinfo (documentation system).

## 4.3 RISC-V GNU Toolchain Installation

Build and install the full RISC-V GNU toolchain (GCC, binutils, gdb, newlib). This toolchain is required to cross-compile C programs for the RV64GC target:

```
  $ bash
git clone --recursive https://github.com/riscv/riscv-gnu-toolchain
cd riscv-gnu-toolchain
./configure --prefix=/opt/riscv
```

```
make
```

This produces riscv64-unknown-elf-gcc, riscv64-unknown-elf-objdump, riscv64-unknown-elf-nm, and riscv64-unknown-elf-addr2line, all of which are used throughout the pipeline.

- Build time is approximately 30–60 minutes, depending on the CPU. Use 'make -j$(nproc)' to parallelize.
- Installation prefix /opt/riscv must be writable. Use sudo or change the prefix to a home directory.

## 4.4 CMake Installation

CMake is required for building several components, including Verilator and LLVM-based tools:

```
  $ bash
sudo apt install cmake
```

## 4.5 Python Virtual Environment Setup

A dedicated Python 3 virtual environment isolates all Python dependencies for the trace pipeline from the system Python installation:

```
  $ bash
sudo apt install python3-full
python3 -m venv ~/ENV_NAME
source ~/ENV_NAME/bin/activate
```

Always activate this environment before running any pipeline scripts. The environment name (ENV_NAME) can be changed to any meaningful identifier, such as 'cva6-trace-env'.

## 4.6 Environment Variable Configuration

These environment variables must be set in every terminal session (or added to ~/.bashrc for persistence):

```
  $ bash
export RISCV=/opt/riscv
export PATH=$RISCV/bin:$PATH
```

$RISCV points to the toolchain installation prefix. Adding $RISCV/bin to PATH makes all riscv64-unknown-elf-* commands available system-wide.

## 4.7 Additional Tool Dependencies

Install help2man (needed for documentation generation) and device-tree-compiler (needed by the CVA6 hardware model):

```
  $ bash
sudo apt-get install help2man device-tree-compiler
```

## 4.8 RISC-V DV Python Requirements

Install Python packages required by the CVA6 digital verification infrastructure. These are needed for running the test regressions and tracing reference flows:

```
  $ bash
cd /cva6
pip3 install wheel
pip3 install -r verif/sim/dv/requirements.txt
pip3 install make
```

## 4.9 Spike ISA Simulator Installation

Install the customized version of Spike that includes CVA6-specific hooks and the post-ISS trace interface:

```
  $ bash
bash verif/regress/install-spike.sh
```

If the automated script fails or Spike is missing from the tools directory, perform manual installation:

```
  $ bash
cd /cva6/verif/core-v-verif/vendor/riscv/riscv-isa-sim
mkdir -p build && cd build
./configure --prefix=$HOME/cva6/tools/spike
make
make install
```

## 4.10 Verilator RTL Simulator Installation

```
  $ bash
bash verif/regress/install-verilator.sh
```

After installation, create a convenience symlink so tool path resolution works correctly:

```
  $ bash
cd /cva6/tools
ln -s verilator-v5.008 verilator
```

## 4.11 Installation Verification

Verify that both Spike and Verilator are correctly installed:

```
  $ bash
cd cva6/tools
ls
```

You should see spike, verilator (or verilator-v5.008), and their associated bin/ directories. If either is missing, re-run the corresponding install script above.

## 4.12 Complete Tool Path Configuration

Configure all tool paths for the full pipeline, RISC-V toolchain, Verilator, and Spike:

```
  $ bash
export
PATH=$RISCV/bin:$HOME/cva6/tools/verilator-v5.008/bin:$HOME/cva6/tools/spik
e/bin:$PATH
```

## 4.13 CVA6 Environment Initialization

Source the CVA6 simulation setup script and set the simulator target:

```
  $ bash
cd /cva6
source verif/sim/setup-env.sh
export DV_SIMULATORS=veri-testharness
```

The setup-env.sh script configures Python paths, regression framework paths, and RISC-V ISA extension settings required by the verification infrastructure.

## 4.14 Smoke Test Execution

Run the smoke test regression to verify that the complete setup is working. This compiles and simulates the hello_world test on CVA6:

```
  $ bash
bash verif/regress/smoke-tests-cv64a6_imafdc_sv39.sh
```

A PASSED result in simulation.log confirms that GCC compilation, Verilator simulation, and the test harness are all functioning correctly.

## 4.15 Verilator Build for Trace Generation

Compile the CVA6 design with Verilator in TRACE_FAST mode, which enables the generation of .dasm trace files required by the System Trace pipeline:

```
  $ bash
make verilate target=cv64a6_imafdc_sv39 TRACE_FAST=1
```

TRACE_FAST=1 enables the lightweight ASCII trace format (as opposed to VCD), which produces the trace_hart_0.dasm and trace_rvfi_hart_00.dasm files consumed by gen_system_trace2.py.

## 4.16 VCD Waveform Generation (Optional)

For detailed waveform analysis, generate a VCD file and convert it to the faster FST format:

```
  $ bash
../../work-ver/Variane_testharness -v VCD_FILE.vcd simulation.elf >
simulation.log 2>&1


# Convert to FST for faster loading in GTKWave
vcd2fst VCD_FILE.vcd FST_FILE.fst


# Open in GTKWave for visual analysis
gtkwave trace.fst
```

## 4.17 Performance Notes

Running Verilator simulation on large programs can exhaust the default stack size. Apply these settings before running:

```
  $ bash
ulimit -s unlimited
export NUM_JOBS=N    # Set N to the number of physical CPU cores
```

# 5. Master Pipeline: run_trace_pipeline1.sh

The master pipeline script is the backbone of the entire trace profiling system. It is a fully automated, 11-step Bash script that orchestrates every stage of the workflow, from compiling the C firmware all the way to generating the final unified profiling report. The script is designed to be industry-ready: it validates all dependencies before starting, handles errors gracefully, and continues where possible when optional components (like RTL cache data) are not available.

### File Location

Path: /mnt/d/trace-project/scripts/run_trace_pipeline1.sh

Execute with: bash run_trace_pipeline1.sh

All outputs are written to: /mnt/d/trace-project/workspace/pipeline_out/

## 5.1 Configuration Block

At the top of the script, all user-configurable paths are defined in a dedicated USER CONFIG section. This makes the script portable; changing the target C file or workspace directory requires editing only this block:

```
  $ bash
TARGET_C=/mnt/d/riscv-work/cva6/verif/tests/custom/hello_world/fibonacci.c
ELF_NAME=simulation.elf
WORKSPACE=/mnt/d/trace-project/workspace/pipeline_out
MAX_CYCLES=0    # 0 = unlimited
```

## 5.2 Tool Path Configuration

All tool binaries are explicitly configured to avoid relying on ambient PATH settings:

```
  $ bash
export SPIKE_ROOT=/mnt/d/riscv-work/cva6/tools/spike
export VERILATOR_ROOT=/mnt/d/riscv-work/cva6/tools/verilator
export RISCV=/usr
export PATH=$RISCV/bin:$VERILATOR_ROOT/bin:$SPIKE_ROOT/bin:$PATH
export DV_SIMULATORS=veri-testharness

CVA6_DIR=/mnt/d/riscv-work/cva6
GCC=/usr/bin/riscv64-unknown-elf-gcc
VERILATOR_BIN=$CVA6_DIR/work-ver/Variane_testharness

POSTISS=/mnt/d/riscv-work/riscv-trace-spec/referenceFlow/build_te/post-iss/
post-iss
ENCODER=/mnt/d/riscv-work/riscv-trace-spec/referenceFlow/scripts/encoder_mo
del.py
DECODER=/mnt/d/riscv-work/riscv-trace-spec/referenceFlow/scripts/decoder_mo
del.py
```

## 5.3 Step 0: Pre-Flight Validation

Before executing any real work, the script performs comprehensive validation of every required file, tool, and directory. This prevents cryptic mid-pipeline failures:

```
  $ bash / python3
check_file "$TARGET_C"
check_tool "$GCC"
check_tool spike
check_tool python3
check_dir  "$CVA6_DIR"
check_file "$VERILATOR_BIN"
check_file "$POSTISS"
check_file "$ENCODER"
check_file "$DECODER"
check_file "$LINKER"
check_file "$SCRIPT_DIR/gen_system_trace2.py"
check_file "$SCRIPT_DIR/func_exec_time.py"
check_file "$SCRIPT_DIR/correlate_misses.py"
check_file "$SCRIPT_DIR/finalize_profiling.py"

if [ "$ERRORS" -gt 0 ]; then
    echo "[FATAL] $ERRORS validation error(s) found. Fix above before
running."
    exit 1
fi
```

## 5.4 Step 1: CVA6 Environment Sourcing

```
  $ bash
source "$CVA6_DIR/verif/sim/setup-env.sh"
```

Loads all CVA6-specific environment variables, Python paths, and simulator configuration required for later steps.

## 5.5 Step 2: Compile C to ELF

Cross-compiles the target C program into a bare-metal RISC-V ELF binary. The compilation includes CVA6-specific runtime startup files (crt.S, syscalls.c) and the CVA6 linker script:

```
  $ bash
```

```
$GCC "$TARGET_C" \
    -I"$DV_INC" \
    -T"$LINKER" \
    -static -mcmodel=medany -fvisibility=hidden -nostdlib -nostartfiles -g
\
    "$COMMON/syscalls.c" \
    "$COMMON/crt.S" \
    -I"$ENV_INC" -I"$COMMON" \
    -lgcc -o "$ELF_NAME" \
    -march=rv64gc_zba_zbb_zbs_zbc_zbkb_zbkx_zkne_zknd_zknh -mabi=lp64d
```

Key flags: -nostdlib -nostartfiles (bare-metal, no standard library), -mcmodel=medany (medium memory model for CVA6 address range), -g (include DWARF debug info for ELF symbol resolution), -march= (full extension set matching CVA6 hardware).

After compilation, the script validates that the ELF exists and is larger than 100 bytes to catch silent compilation failures.

## 5.6 Step 3: Spike ISA Simulation

Runs the compiled ELF through the Spike RISC-V ISA simulator with commit logging enabled. Every retired instruction is logged with its PC and register state:

```
  $ bash
spike --isa=rv64gc_zba_zbb_zbs_zbc_zbkb_zbkx_zkne_zknd_zknh \
    -l --log-commits \
    "$WORKSPACE/$ELF_NAME" \
    > spike_raw.log 2>&1
```

The raw log is then processed with AWK to extract only user-mode (privilege level 3) instructions at CVA6's base address (0x80000000+), producing the clean spike_pc_trace.txt:

```
  $ bash
echo "VALID, ADDRESS, INSN, PRIVILEGE, EXCEPTION, ECAUSE, TVAL, INTERRUPT"
> spike_pc_trace.txt
grep "^core" spike_raw.log | grep " 3 " | awk '
/^core.*0x00000000800/ {
    priv=$3; addr=$4; insn=$5
    printf "1,%s,%s,%s,0,0,0,0\n", addr, insn, priv
}' >> spike_pc_trace.txt
```

## 5.7 Step 4: Verilator RTL Simulation

Runs the ELF through the Verilator-compiled CVA6 RTL model. This produces cycle-accurate simulation outputs, including the .dasm trace files:

```
  $ bash
"$VERILATOR_BIN" "$WORKSPACE/$ELF_NAME" > simulation.log 2>&1
```

Output files: simulation.log (pass/fail status), trace_hart_0.dasm (cycle-accurate PC trace), trace_rvfi_hart_00.dasm (RISC-V Formal Interface state trace with register and memory values).

## 5.8 Step 5: Write Configuration Files

Generates four configuration files needed by the E-Trace reference flow tools. These files describe the hardware configuration of the trace encoder/decoder:

- my_hardware.scf: instruction trace hardware capabilities (address width, privilege levels, filter support)
- data_hardware.scf: data trace hardware capabilities (data width, address width)
- User_encoder.cfg: encoder parameters (file stem, ELF path, codec settings)
- User_data.cfg: data encoder parameters (compression settings)

## 5.9 Step 6: Post-ISS Processing

The post-iss tool converts the Spike PC trace into the encoder_input format required by the RISC-V trace reference encoder. This step adds instruction type annotations (call, return, branch, etc.):

```
  $ bash
"$POSTISS" -c "$SCF" -i spike_pc_trace.txt


cp spike_pc_trace.encoder_input spike_pc_trace_input
```

## 5.10 Step 7: E-Trace Encoding and Decoding

This is the core E-Trace pipeline. First, the encoder compresses the instruction trace into N-Trace packets. Then the decoder reconstructs the original PC trace from those packets:

```
  $ bash
# Encoding
python3 "$ENCODER" -c "$SCF" -u "$USER_CFG"


# Decoding
python3 "$DECODER" -c "$SCF" -u "$USER_CFG" -i
spike_pc_trace_input.te_inst_raw
```

After decoding, the pipeline performs automatic verification by comparing the original Spike PC trace against the decoded trace. A match confirms lossless round-trip compression:

```
  $ bash
if diff spike_check.txt decoder_check.txt > /dev/null 2>&1; then
    echo "[OK] E-Trace MATCH — decode verified"
else
    DIFF_COUNT=$(diff spike_check.txt decoder_check.txt | grep '^[<>]' | wc
-l)
    echo "[WARN] E-Trace MISMATCH ($DIFF_COUNT differing lines)"
fi
```

## 5.11 Step 8: System Trace Generation

Invokes gen_system_trace2.py to merge the two Verilator .dasm trace files into a single, human-readable system_trace.txt. This step requires both .dasm files from the Verilator simulation:

```
  $ bash
python3 "$SCRIPT_DIR/gen_system_trace2.py" \
    --hart  "$HART_DASM" \
    --rvfi  "$RVFI_DASM" \
    --elf   "$WORKSPACE/$ELF_NAME" \
    --out   "$SYS_TRACE_TXT" \
    --collapse-syscall
```

If the .dasm files are not present (e.g., Verilator was not run), this step is gracefully skipped with a warning, and all downstream steps that depend on it are also skipped.

## 5.12 Step 9: Function Execution Time

Runs func_exec_time.py on the merged system trace to compute per-function profiling statistics:

```
  $ bash
python3 "$SCRIPT_DIR/func_exec_time.py" \
    --trace  "$SYS_TRACE_TXT" \
    --outdir "$WORKSPACE"
```

Outputs: function_exec_time.csv (machine-readable, Eclipse-importable) and function_exec_time.txt (human-readable summary report).

## 5.13 Step 10: RTL Cache Miss Correlation

If icache_misses.log and dcache_misses.log are present (generated by Verilator with custom RTL instrumentation), run correlate_misses.py to map cache misses to source functions:

```
  $ bash
python3 "$SCRIPT_DIR/correlate_misses.py" \
    --icache "$ICACHE_LOG" \
    --dcache "$DCACHE_LOG" \
    --elf    "$WORKSPACE/$ELF_NAME" \
    --outdir "$WORKSPACE"
```

This step is entirely optional. If the cache log files are not present, the step is skipped without failing the pipeline. Cache data enriches the unified report but is not required.

## 5.14 Step 11: Unified Profiling Report

The final step invokes finalize_profiling.py to merge all available data sources into the unified report:

```
  $ bash
python3 "$SCRIPT_DIR/finalize_profiling.py" \
    --systrace "$WORKSPACE/function_exec_time.csv" \
    --systxt   "$WORKSPACE/system_trace.txt" \
    $RTL_ARG \
    --elf      "$WORKSPACE/$ELF_NAME" \
    --outdir   "$WORKSPACE"
```

## 5.15 Pipeline Output Summary

| Output File | Description |
|---|---|
| **simulation.elf** | Cross-compiled RISC-V firmware binary |
| **spike_raw.log** | Full Spike ISA simulation log with all commit records |
| **spike_pc_trace.txt** | Cleaned PC trace (CSV format) ready for E-Trace encoder |
| **simulation.log** | Verilator RTL simulation pass/fail log |
| **spike_pc_trace_input.decoder.trace** | E-Trace decoded PC trace (verified against original) |
| **system_trace.txt** | Merged hart + RVFI system trace with function annotations |
| **function_exec_time.csv** | Per-function cycle timing in machine-readable CSV |
| **function_exec_time.txt** | Human-readable function profiling report |
| **rtl_profiling_report.csv** | RTL-level cache miss data per function (if available) |

| | |
|---|---|
| **unified_profiling_report.csv** | Complete unified profiling report (Eclipse-ready CSV) |
| **unified_profiling_summary.txt** | Complete unified profiling report (human-readable TXT) |
| | |

# 6. Script: gen_system_trace2.py

### File Info

Path: /mnt/d/trace-project/scripts/gen_system_trace2.py
Purpose: Merge trace_hart_0.dasm + trace_rvfi_hart_00.dasm into system_trace.txt
Python Version: 3.x (no external dependencies)

gen_system_trace2.py is the System Trace decoder. The CVA6 Verilator model produces two separate trace files: trace_hart_0.dasm contains cycle-accurate timing information for each executed instruction, while trace_rvfi_hart_00.dasm contains the full RISC-V Formal Interface state, register writes, and memory accesses. Neither file alone gives a complete picture. This script merges them into a single, annotated system trace.

## 6.1 Input Files

| Input | Description |
|---|---|
| **trace_hart_0.dasm** | Cycle-accurate PC trace from CVA6 Verilator model. Format: CYCLE PC OPCODE. Contains a hardware cycle counter for every retired instruction. |
| **trace_rvfi_hart_00.dasm** | RISC-V Formal Interface trace. Contains register destination writes (which register was updated and to what value) and memory accesses (load/store address and data) for every instruction. |
| **simulation.elf (--elf)** | Optional. Used by riscv64-unknown-elf-nm to extract symbol table. Enables mapping of PC addresses to function names. |

## 6.2 Output Format: system_trace.txt

The output file has seven fixed-width columns separated by two or more spaces:

```
1619  0x000000008000306a  jal           CALL    main                  –
1624  0x0000000080003000  c.arith       EXEC    fibonacci             –
1626  0x0000000080003002  c.misc        EXEC    fibonacci             –
1627  0x0000000080003004  c.misc        EXEC    fibonacci             –
1628  0x0000000080003006  c.misc        EXEC    fibonacci             –
1628  0x0000000080003008  c.misc        EXEC    fibonacci             –
1629  0x000000008000300a  c.mv          EXEC    fibonacci             a5=0x0000000000000000
1631  0x000000008000300c  sw            STORE   fibonacci             –
1637  0x0000000080003010  lw            LOAD    fibonacci             a5=0x0000000000000000
1638  0x0000000080003014  addiw         EXEC    fibonacci             a4=0x0000000000000000
1639  0x0000000080003018  c.arith       EXEC    fibonacci             a5=0x0000000000000001
1640  0x000000008000301a  blt           BRANCH  fibonacci             –
1648  0x000000008000301e  lw            LOAD    fibonacci             a5=0x0000000000000000
1648  0x0000000080003022  c.arith       EXEC    fibonacci             –
1652  0x000000008000304a  c.mv          EXEC    fibonacci             a0=0x0000000000000000
```

## 6.3 ELF Symbol Loading (pc_to_func)

The script uses riscv64-unknown-elf-nm to extract all text/weak symbols from the ELF. These are sorted by address. For each instruction PC, a binary search finds the nearest symbol with an address less than or equal to the PC, which is the function that contains that instruction:

```
  python3
def pc_to_func(pc: int, symbols: list) -> str:
    lo, hi = 0, len(symbols) - 1
    result = '?'
    while lo <= hi:
        mid = (lo + hi) // 2
        if symbols[mid][0] <= pc:
            result = symbols[mid][1]
            lo = mid + 1
        else:
            hi = mid - 1
    return result
```

## 6.4 Opcode to Mnemonic Decoder

Rather than relying on an external disassembler for each instruction, the script includes a compact inline opcode decoder. It handles both compressed (16-bit, C. extension) and standard (32-bit) RISC-V instructions:

- Compressed instructions (bits [1:0] != 0b11): decoded by quadrant and function bits
- Standard instructions: decoded by opcode (bits [6:0]), funct3 (bits [14:12]), and funct7 (bits [31:25])
- Covers all base integer, load/store, branch, multiplication/division, and CSR instructions

## 6.5 Event Classification

Every instruction is classified into one of seven event types based on its mnemonic:

| Event | Instructions |
|---|---|
| **SYSCALL** | ecall, ebreak — system call or breakpoint |
| **BRANCH** | beq, bne, blt, bge, bltu, bgeu, c.beqz, c.bnez |
| **CALL** | jal, c.jal, c.j — function calls and unconditional jumps |
| **RET** | jalr, c.jalr, c.jr, ret — returns and indirect jumps |
| **LOAD** | lb, lh, lw, ld, lbu, lhu, lwu, c.lw, c.ld, and variants |

| | |
|---|---|
| **STORE** | sb, sh, sw, sd, c.sw, c.sd and variants |
| **EXEC** | All other instructions (arithmetic, logical, CSR, etc.) |

## 6.6 Noise Function Collapsing (--collapse-syscall)

The --collapse-syscall flag enables intelligent noise reduction. CVA6 test programs include significant startup and library code (printf internals, memset, TLS initialization) that would otherwise flood the trace. The script collapses repetitive instructions in known noise functions while leaving all user functions completely intact:

```
python3
NOISE_FUNCTIONS = {
    # Standard library internals
    'syscall', 'memset', 'memcpy', 'memcmp', 'memmove',
    'strlen', 'strcpy', 'strcmp',
    # printf internals
    'vprintfmt', 'putchar', 'puts', 'printf', 'sprintf',
    'printnum', 'getint',
    # startup / TLS
    '_init', 'init_tls', 'thread_entry',
    # unknown
    '?',
}
```

Critical design decision: any function NOT in NOISE_FUNCTIONS is never collapsed — even if the same PC repeats many times. This ensures user code (fibonacci, main, or any custom function) always produces complete trace output regardless of its execution pattern.

## 6.7 Hart Cycle Lookup

The RVFI trace provides instruction sequence and register state, but not cycle numbers. The hart trace provides cycle numbers. The merge works by maintaining a per-PC ordered queue from the hart trace. For each RVFI record, the corresponding cycle is consumed from the queue for that PC address:

```
  python3
hart_queue = {pc: list(entries) for pc, entries in hart_table.items()}
hart_pos   = defaultdict(int)

# For each RVFI record:
if pc in hart_queue:
    idx = hart_pos[pc]
    if idx < len(hart_queue[pc]):
        cycle = hart_queue[pc][idx][0]
        hart_pos[pc] += 1
```

# MEM_ACCESS Parsing Fix

## Bug

The `RVFI_MEM_RE` regular expression was originally written assuming that the RVFI trace format explicitly contained a memory access direction token (`r`/`w`). However, CVA6's actual RVFI trace output does not include such a token. Consequently, the regex failed to match any memory access entries, causing the `MEM_ACCESS` column to display, for all LOAD and STORE instructions.

## Root Cause

The actual CVA6 RVFI trace formats are:

# STORE
<pc> (<opcode>) mem <addr> <value>

# LOAD
<pc> (<opcode>) x <reg> <value> mem <addr>

No explicit read/write indicator is present in the trace. Instead, the access type must be inferred from the instruction context:

- If a register write (`x <reg> <value>`) appears before the `mem` field, the operation is a LOAD (memory read).
- Otherwise, the operation is a STORE (memory write).

## Fix

The memory access regex was updated to match the actual CVA6 RVFI format, and additional logic was introduced to infer access direction dynamically.

### Previous Implementation (Incorrect)

```
RVFI_MEM_RE = re.compile(
    r'(mem|MA)\s+(0x[0-9a-fA-F]+)\s+(r|w|R|W)\s+(0x[0-9a-fA-F]+)'
)
```

### Updated Implementation (Correct)

```
RVFI_MEM_RE = re.compile(
    r'mem\s+(0x[0-9a-fA-F]+)(?:\s+(0x[0-9a-fA-F]+))?'
)
```

### Direction Inference Logic

```
rw = "R" if re.search(
    r'\bx\s+\d+\s+0x',
    line[:line.index('mem')]
) else "W"
```

## Verification

After applying the fix, memory accesses were correctly extracted and displayed in the profiling output.

Example outputs:

```
iqra@DESKTOP-35FRKQ6:/mnt/d/trace-project/workspace/pipeline_out$ grep "factorial" system_trace.txt | grep "STORE\|LOAD"
      1609  0x000000008000300a  sw            STORE     factorial            -
       W@0x80025f5c=0x00000005
      1614  0x000000008000300e  lw            LOAD      factorial            a5=0x0000000000000005
       -
      1633  0x0000000080003020  lw            LOAD      factorial            a5=0x0000000000000005
       -
      1644  0x000000008000300a  sw            STORE     factorial            -
       W@0x80025f3c=0x00000004
      1650  0x000000008000300e  lw            LOAD      factorial            a5=0x0000000000000004
       -
      1661  0x0000000080003020  lw            LOAD      factorial            a5=0x0000000000000004
       -
      1672  0x000000008000300a  sw            STORE     factorial            -
       W@0x80025f1c=0x00000003
      1688  0x000000008000300e  lw            LOAD      factorial            a5=0x0000000000000003
       -
      1694  0x0000000080003020  lw            LOAD      factorial            a5=0x0000000000000003
       -
      1705  0x000000008000300a  sw            STORE     factorial            -
       W@0x80025efc=0x00000002
      1721  0x000000008000300e  lw            LOAD      factorial            a5=0x0000000000000002
       -
      1727  0x0000000080003020  lw            LOAD      factorial            a5=0x0000000000000002
       -
      1738  0x000000008000300a  sw            STORE     factorial            -
       W@0x80025edc=0x00000001
      1754  0x000000008000300e  lw            LOAD      factorial            a5=0x0000000000000001
       -
      1778  0x0000000080003030  lw            LOAD      factorial            a4=0x0000000000000002
       -
      1795  0x0000000080003030  lw            LOAD      factorial            a4=0x0000000000000003
```

The recursive stack-frame pushes and pops inside the `factorial()` function are now fully visible in the `MEM_ACCESS` column, confirming that LOAD and STORE operations are being parsed correctly.

# 7. Script: func_exec_time.py

### File Info

Path: /mnt/d/trace-project/scripts/func_exec_time.py
Purpose: Compute per-function execution time, CPI, and memory intensity from system_trace.txt

.

func_exec_time.py is the primary profiling engine. It reads system_trace.txt and performs a detailed statistical analysis of every function in the trace. The key challenge in profiling is correctly attributing CPU cycles to the function that was actually responsible for them, not to a caller that was simply waiting for a callee to finish. This script solves this with a call-stack tracking algorithm.

## 7.1 Trace Parsing

The parser reads system_trace.txt line by line, splitting on two-or-more-space boundaries (the fixed-width column separator). It extracts cycle number, event type, function name, register delta, and memory address:

```
python3
def parse_trace(path):
    rows = []
    header_done = False
    with open(path, 'r', errors='replace') as f:
        for line in f:
            stripped = line.strip()
            if not header_done:
                if stripped.startswith('CYCLE') or
stripped.startswith('---'):
                    header_done = True
                continue
            parts = re.split(r'\s{2,}', stripped)
            if len(parts) < 5: continue
            cycle = int(parts[0])
            event = parts[3].strip().upper()
            func  = parts[4].strip()
            ...
```

Memory addresses are extracted from the REG_DELTA column (column index 5) using a regex pattern matching '0x[hex]+'. This is the corrected approach — early versions incorrectly read from the MEM_ACCESS column, which is typically '-'.

## 7.2 Self-Cycle Computation (Call Stack Algorithm)

This is the core algorithm. Instead of measuring total elapsed time between function entry and exit (which would double-count callee time), the script tracks a call stack and attributes the cycle gap between consecutive instructions to the function at the top of the stack:

```python3
call_stack = []

for i, row in enumerate(rows):
    func  = row['func']
    cycle = row['cycle']

    # Self cycles: the gap before the NEXT instruction belongs to
    # whoever is currently at the top of the call stack
    if i + 1 < len(rows):
        gap = rows[i + 1]['cycle'] - cycle
        if gap > 0 and call_stack:
            stats[call_stack[-1]]['self_cycles'] += gap
```

When the current instruction belongs to a different function than the stack top, the stack is updated: if the new function is already deeper in the stack (a return), frames are popped until it is reached; if it is new (a call), it is pushed.

## 7.3 Computed Metrics

| Column | Formula | Notes |
|---|---|---|
| TOTAL_CYCLES(Self Cycles) | self_cycles = Σ gapwhere top-of-stack == this function | Excludes time spent in callees. Primary hotspot metric. |
| PCT_OF_PROGRAM | self_cycles / total_program_cycles × 100 | total_program_cycles = last_cycle − first_cycle in trace |
| CPI | self_cycles / instr_count | Profiling-level CPI, not architectural. High CPI indicates stalls or cache pressure. |
| INSTR_COUNT | count of all rows where func == this function | Total instructions executed across all calls. |
| CALLS | count of distinct entries on the call stack | Approximate and accurate for structured traces. |
| AVG_CYCLES_PER_CALL | self_cycles / calls | Stable formula (v4 fix). Earlier versions used span tracking, which gave impossible values. |
| EXEC / LOAD / STORE / BRANCH | count of rows by event type | Breakdown of instruction types executed in this function. |
| MEM_ACCESSES | load_count + store_count | Total memory operations. |
| UNIQUE_MEM_ADDRS | len(set of distinct addresses seen) | Wide value = large memory footprint. |

| Column | Formula | Notes |
|---|---|---|
| MEM_INTENSITY | mem_accesses / self_cycles | Ratio close to 1.0 means nearly every cycle has a memory operation, a strong cache pressure signal. Safe: returns 0.0 if cyc == 0. |

## 7.4 Output Files

The script writes two output files to the specified --outdir:

- function_exec_time.csv: machine-readable CSV with all metrics. This file is consumed by finalize_profiling.py and can be imported directly into Eclipse.
- function_exec_time.txt: human-readable formatted report with two sections: the full function table, and two ranked top-5 lists (hotspots by self_cycles and memory-intensive functions by mem_accesses).

## 7.5 Terminal Preview Output

For quick verification, the script also prints a formatted table of the top 10 functions to the terminal, along with a PCT sanity check (the sum of all function PCT values should be approximately 100%):

```
  Instructions   :        1,118
================================================================================================================

FUNCTION               SELF_CYC      %   CPI  INSTRS  CALLS  AVG_CYC   EXEC   LOAD  STORE  BRANCH  MEM_ACC  UNIQ_ADDR  MEM_INT
----------------------------------------------------------------------------------------------------------------
memset                      844  30.54%  1.89     446      1    844.0     93    217     89      47      306         44   0.3626
vprintfmt                   509  18.42%  2.47     206      1    509.0     59     75     33      39      108         27   0.2122
putchar                     482  17.44%  2.75     175     11     43.8     66     44     33      32       77         20   0.1598
printnum                    302  10.93%  2.75     110      2    151.0     40     30     24      16       54          5   0.1788
getint                      110   3.98%  3.44      32      2     55.0     14     10      4       4       14          2   0.1273
memcpy                       97   3.51%  3.46      28      1     97.0      9      9      5       5       14          3   0.1443
syscall                      89   3.22%  3.56      25      1     89.0      9      5      9       2       14          5   0.1573
printf                       74   2.68%  4.93      15      1     74.0      6      3      5       1        8          3   0.1081
init_tls                     66   2.39%  4.12      16      1     66.0      8      3      2       3        5          2   0.0758
_init                        62   2.24%  3.65      17      1     62.0      7      3      3       4        6          3   0.0968
main                         58   2.10%  2.76      21      1     58.0     14      3      1       3        4          1   0.0690
thread_entry                 36   1.30%  4.50       8      1     36.0      4      1      2       1        3          1   0.0833
fibonacci                    35   1.27%  1.84      19      1     35.0     14      2      1       2        3          1   0.0857

----------------------------------------------------------------------------------------------------------------
```

### 7.5.1 Additional Outputs

```
TOP 5 CPU HOTSPOTS (self cycles):

#   FUNCTION                    SELF_CYC       %    CPI   INSTRS   CALLS
-----------------------------------------------------------------------
1   tohost_exit             1,997,964  99.92%  2.00 998,984       1
2   memset                        846   0.04%  1.90     446       1
3   factorial                     240   0.01%  2.12     113       1
4   _init                         196   0.01%  3.77      52       1
5   memcpy                         92   0.00%  3.29      28       1

TOP 5 MEMORY-INTENSIVE FUNCTIONS
```

It also shows the Top 5 CPU hotspot functions, where tohost_exit dominates at 99.92% self cycles. This is the HTIF busy-wait exit loop (hardware I/O spin), not actual program logic. The real computation is factorial at 0.01% with CPI 2.12, confirming efficient execution. memset/memcpy are standard CRT initialization routines.

```
TOP 5 MEMORY-INTENSIVE FUNCTIONS:

#   FUNCTION                MEM_ACC    LOAD   STORE  UNIQ_ADDR  MEM_INT
-------------------------------------------------------------------------
1   memset                      306     217      89         44   0.3617
2   _init                        22      14       8          5   0.1122
3   factorial                    18      13       5          5   0.0750
4   memcpy                       14       9       5          3   0.1522
5   init_tls                      5       3       2          2   0.0769
```

It also shows the top 5 Memory-Intensive Functions, with memset being the most memory-intensive (MEM_INT: 0.3617) due to CRT buffer initialization. factorial accesses only 5 unique memory addresses; consistent with stack-based recursive computation. Low overall memory pressure across all user functions.

# 8. Script: correlate_misses.py

### File Info

Path: /mnt/d/trace-project/scripts/correlate_misses.py

Purpose: Map RTL-level I-Cache and D-Cache miss addresses to source functions

Inputs: icache_misses.log, dcache_misses.log, simulation.elf

correlate_misses.py bridges the gap between low-level RTL cache behavior and high-level source code. When the CVA6 Verilator model is instrumented to log cache miss events (instruction fetch misses and data access misses), this script reads those logs and uses the firmware ELF's symbol table to determine which function was executing at the time of each miss.

## 8.1 Symbol Table Construction

The script builds a precise function address map using riscv64-unknown-elf-nm with --numeric-sort and --print-size flags. Unlike a simple symbol lookup, this approach captures function size so that the boundaries between adjacent functions are exact:

```
python3
nm_out = subprocess.check_output(
    ['riscv64-unknown-elf-nm', '--numeric-sort', '--print-size', args.elf],
).decode('utf-8', errors='replace')

symbols = []
for line in nm_out.splitlines():
    parts = line.split()
    if len(parts) == 4:
        addr = int(parts[0], 16)
        size = int(parts[1], 16)
        if parts[2].lower() in ('t', 'w'):  # text/weak symbols only
            symbols.append((addr, size, parts[3]))

# For zero-size symbols, use the next symbol's start as the end boundary
for i, (addr, size, name) in enumerate(symbols):
    end = symbols[i+1][0] if (size == 0 and i+1 < len(symbols)) else addr +
max(size, 4)
    func_map[addr] = (name, end)
```

## 8.2 Cache Miss to Function Mapping

Each cache miss log file is a CSV with a pc_hex (for I-Cache) or addr_hex (for D-Cache) column. The script reads every miss entry and maps its address to the containing function using binary search:

```python3
def pc_to_func(pc_str):
    pc = int(pc_str.strip(), 16)
    lo, hi, result = 0, len(sorted_addrs)-1, None
    while lo <= hi:
        mid = (lo+hi)//2
        if sorted_addrs[mid] <= pc:
            result = sorted_addrs[mid]
            lo = mid+1
        else:
            hi = mid-1
    name, end = func_map[result]
    return name if pc < end else '<unknown>'

# Process I-Cache misses
with open(args.icache) as f:
    for row in csv.DictReader(f):
        ic[pc_to_func(row.get('pc_hex','').replace('0x',''))] += 1

# Process D-Cache misses
with open(args.dcache) as f:
    for row in csv.DictReader(f):
        dc[pc_to_func(row.get('addr_hex','').replace('0x',''))] += 1
```

## 8.3 Output Files

- rtl_profiling_report.csv: columns: function, icache_misses, dcache_misses, total_misses. Consumed by finalize_profiling.py.
- rtl_profiling_summary.txt: formatted text report showing total cache miss counts and a per-function ranked table.

```
iqra@DESKTOP-S5PRRQ8:/mnt/d/trace-project/workspace/pipeline_out$
==============================================================
  RTL CACHE MISS REPORT — CVA6 REAL HARDWARE
==============================================================

  Total I-Cache misses : 140
  Total D-Cache misses : 27

Function                         I$Miss    D$Miss     Total
--------------------------------------------------------------
<unknown>                             1        27        28
vprintfmt                            25         0        25
_start                               20         0        20
printnum                             19         0        19
memset                               11         0        11
memcpy                                9         0         9
syscall                               8         0         8
putchar                               8         0         8
getint                                7         0         7
init_tls                              7         0         7
printf                                6         0         6
_init                                 6         0         6
main                                  6         0         6
fibonacci                             4         0         4
thread_entry                          3         0         3

==============================================================
```

# 9. Script: finalize_profiling.py

### File Info

Path: /mnt/d/trace-project/scripts/finalize_profiling.py

Purpose: Merge all data sources into the unified profiling report (CSV + TXT)

finalize_profiling.py is the reporting engine. It reads all available profiling data sources, resolves source file locations using addr2line, merges everything into a single data structure, and writes three output formats. It is designed to degrade gracefully; if cache data or system trace data is unavailable, it produces the best possible report from what is present.

## 9.1 Data Sources

| Source | Role & Handling |
|---|---|
| **function_exec_time.csv (PRIMARY)** | Required. Provides all timing metrics: self_cycles, PCT, CPI, INSTR_COUNT, CALLS, AVG_CYCLES_PER_CALL, EXEC, LOAD, STORE, BRANCH. |
| **system_trace.txt (SUPPLEMENTARY)** | Optional. Re-parsed to extract MEM_ACCESSES and UNIQUE_MEM_ADDRS per function. Addresses extracted from the REG_DELTA column (v6 fix — earlier versions incorrectly used the MEM_ACCESS column, which is '-'). |
| **rtl_profiling_report.csv (OPTIONAL)** | Optional. Provides ICACHE_MISSES, DCACHE_MISSES, and TOTAL_CACHE_MISSES per function. If absent, these columns show N/A. |
| **simulation.elf (ELF)** | Required for addr2line source location resolution. Maps function names to source file:line_number using riscv64-unknown-elf-addr2line. |

## 9.2 Source Location Resolution

For every function in the report, the script resolves the source file and line number using addr2line:

```
python3
def addr2line_single(elf_path, addr_hex):
    out = subprocess.check_output(
        ['riscv64-unknown-elf-addr2line', '-e', elf_path, '-f', '-s',
f'0x{addr_hex}'],
        text=True
    )
    lines = [l.strip() for l in out.strip().splitlines() if l.strip()]
    if len(lines) >= 2:
        loc = lines[1]
        return 'unknown' if loc.startswith('?') or loc == ':0' else loc
    return 'unknown'
```

The result is a source location like 'fibonacci.c:42' that appears in the SOURCE_LOCATION column of the unified report, making it directly actionable for engineers.

## 9.3 Bug Fix: UNIQUE_MEM_ADDRS

A significant bug was fixed in v6: the load_systxt() function was reading memory addresses from the MEM_ACCESS column (parts[6]) of system_trace.txt. However, the MEM_ACCESS column is almost always '-'. The actual memory addresses appear in the REG_DELTA column (parts[5]) in the format 'a5=0x80026020'. The fix:

```
python3
# WRONG (v5): read from MEM_ACCESS column
mem_field = parts[6].strip() if len(parts) > 6 else '-'

# CORRECT (v6): read from REG_DELTA column
reg_delta = parts[5].strip() if len(parts) > 5 else '-'
m = ADDR_RE.search(reg_delta)  # regex: 0x[0-9a-fA-F]+
if m:
    data[func]['addrs'].add(m.group(0).lower())
```

## 9.4 Bootstrap Function Tagging

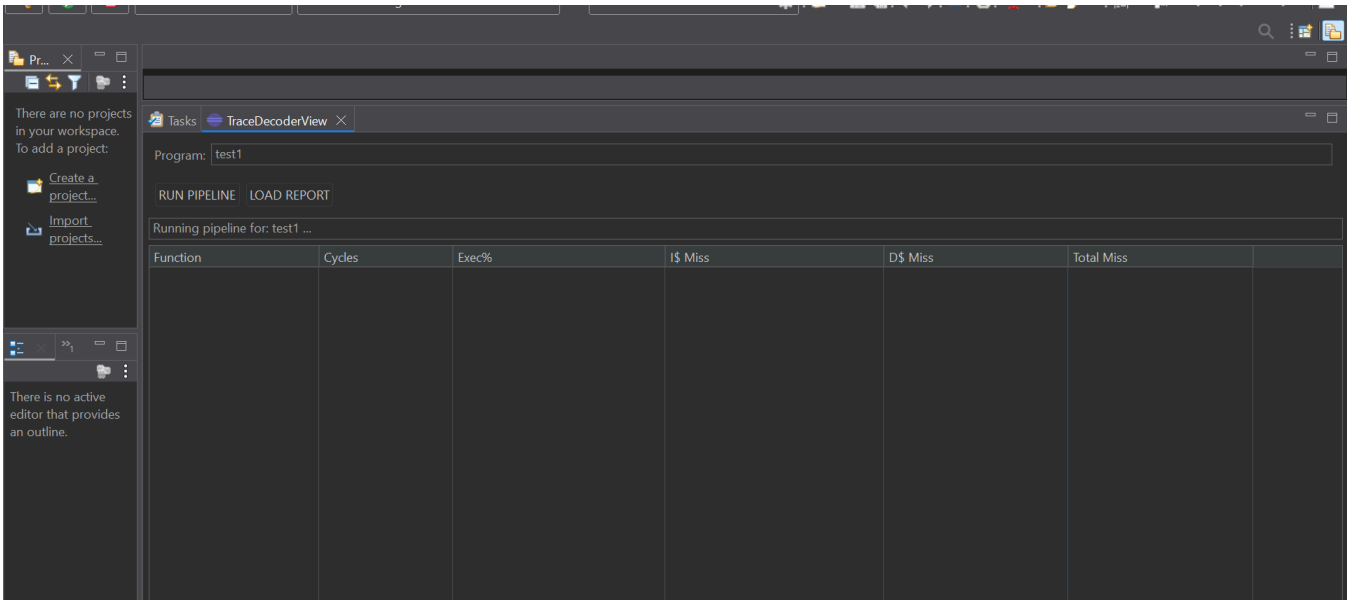
Functions that are part of the C runtime startup (not user code) are automatically tagged with [SYS] prefix in the report for clarity:

```
python3
```

```
BOOTSTRAP = {'_start', '_init', 'init_tls', 'thread_entry',
             '__libc_start_main', 'crt0', '_fini'}

def to_display(fn):
    return f'[SYS] {fn}' if fn in BOOTSTRAP else fn
```

## 10. ECLIPSE INTEGRATION:




The Eclipse integration is the final layer of the system. It has two distinct parts:
(1) the master pipeline script automates the entire build-simulate-profile-launch sequence, ending with a live Eclipse window that has already loaded all results; and
(2) The TraceDecoderView Eclipse plugin provides four structured views: Unified Profiling, System Trace, E-Trace, and Cache Misses that render the pipeline outputs as interactive tables inside the IDE.

## 10.1 Pipeline Automation: Trigger File and Eclipse Launch

After the Unified Profiling Report is written in Step 11, the pipeline performs two final automation steps.

## Step 1: Write the Eclipse File Trigger

The pipeline writes a Unix timestamp into a file named pipeline_done.trigger in the pipeline output directory:

```
echo "[TRIGGER] Writing pipeline_done.trigger..."
date +%s > "$TRIGGER_FILE"
echo "[OK] Trigger written → Eclipse will auto-load table"
```

This trigger file is the communication channel between the pipeline and the Eclipse plugin. The TraceDecoderView runs a background watcher thread that polls this file every 2 seconds. When the file is created or its modification timestamp advances, the plugin reloads all four data tables automatically, without any manual action from the user.

## Step 2: Launch Eclipse

After writing the trigger, the pipeline launches Eclipse using the Java launcher directly:

```
"$JAVAW" \
    -classpath "$LAUNCHER_JAR" \
    org.eclipse.equinox.launcher.Main \
    -launcher "C:\\eclipse\\eclipse.exe" \
    -product org.eclipse.platform.ide \
    -data "$RUNTIME_DATA" \
    -configuration "$RUNTIME_CFG" \
    -dev "$RUNTIME_DEV" \
    -os win32 -ws win32 -arch x86_64 -nl en_US &
echo "[OK] Eclipse launched — table auto-loading pipeline results!"
```

Key parameters:
- LAUNCHER_JAR: path to org.eclipse.equinox.launcher_*.jar inside the Eclipse installation
- RUNTIME_DATA: the Eclipse workspace directory where the plugin is active
- RUNTIME_CFG: Eclipse runtime configuration directory
- RUNTIME_DEV: dev.properties file listing plugin classpaths (used during plugin development)
- The & at the end launches Eclipse as a background process, so the terminal is returned immediately

This design means a single command (bash run_trace_pipeline1.sh) takes a C source file all the way from compilation through Verilator simulation, E-Trace encoding/decoding, system trace generation, profiling, and finally opens Eclipse with all results already loaded in the plugin views.

## 10.2 Eclipse Plugin: com.trace.decoder

The Eclipse plugin is implemented as a ViewPart under the plugin ID com.trace.decoder. The main class is TraceDecoderView.java, located at:

```
com.trace.decoder/src/com/trace/decoder/views/TraceDecoderView.java
```

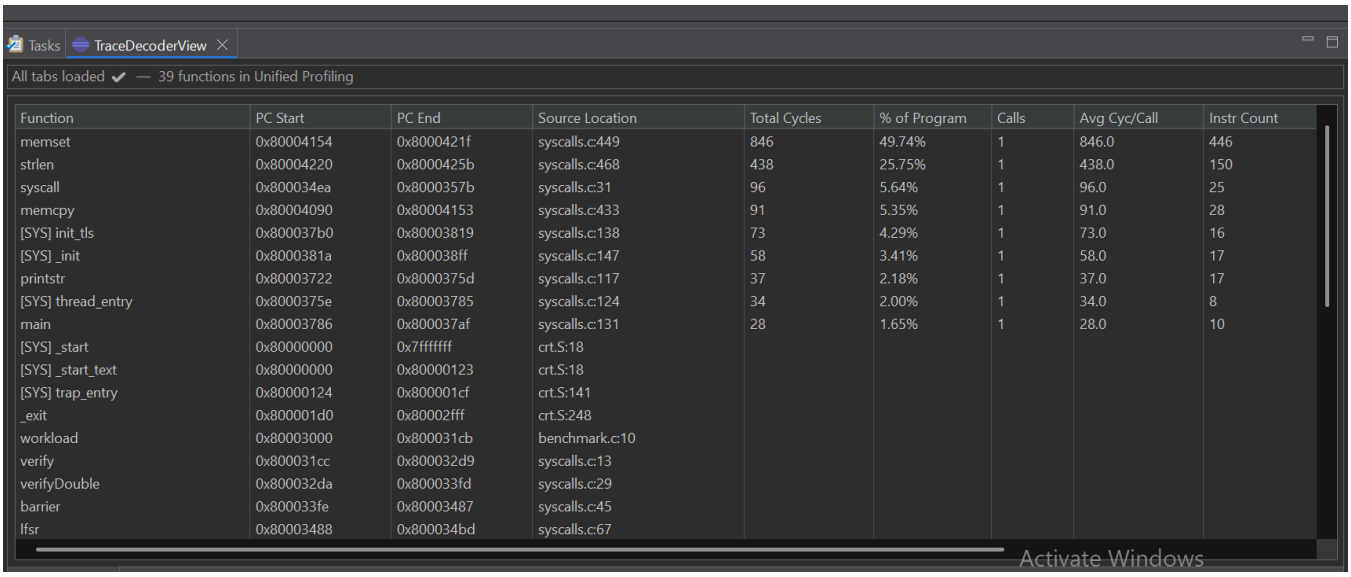
The plugin reads all pipeline output files directly from the Windows filesystem (D:\trace-project\workspace\pipeline_out\) through the WSL-mounted path.

## 10.2.1 Eclipse Plugin Structure

The view is organized as a single CTabFolder containing four tabs. Each tab is backed by a scrollable SWT Table widget with column headers, full-row selection, and horizontal/vertical scrollbars.

### Tab 1: Unified Profiling

Reads: unified_profiling_report.csv
Columns: Function | PC Start | PC End | Source Location |
Total Cycles | % of Program | Calls | Avg Cyc/Call |
Instr Count | I$ Misses | D$ Misses | Total Cache Miss

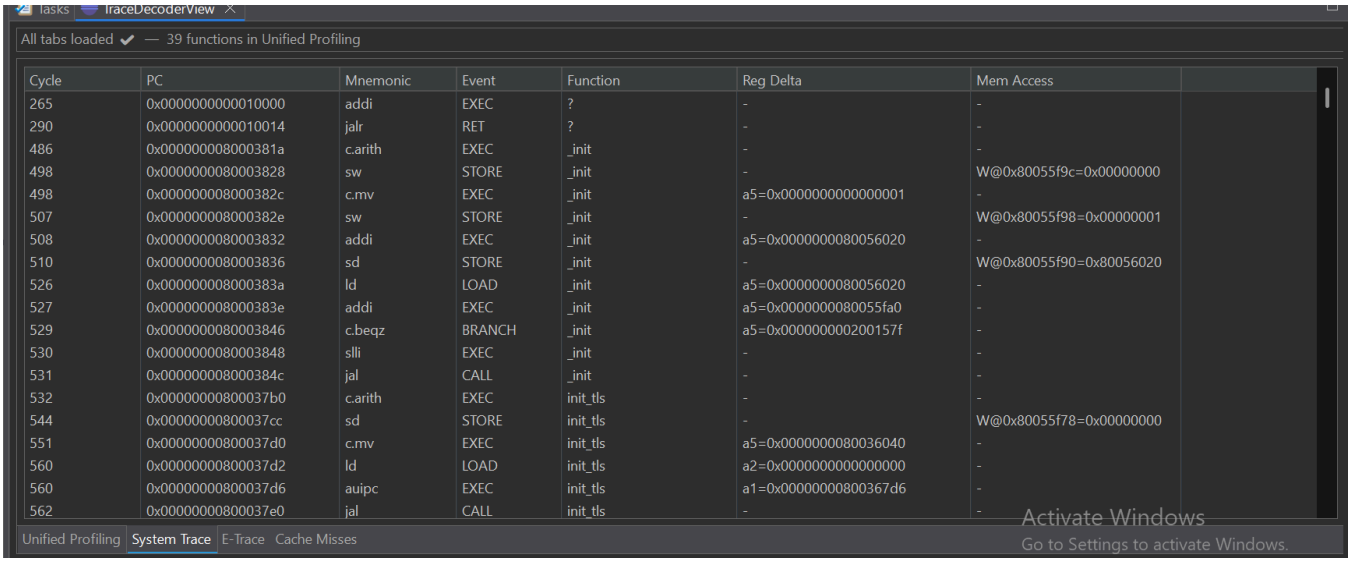



| Function | PC Start | PC End | Source Location | Total Cycles | % of Program | Calls | Avg Cyc/Call | Instr Count |
|---|---|---|---|---|---|---|---|---|
| memset | 0x80004154 | 0x8000421f | syscalls.c:449 | 846 | 49.74% | 1 | 846.0 | 446 |
| strlen | 0x80004220 | 0x8000425b | syscalls.c:468 | 438 | 25.75% | 1 | 438.0 | 150 |
| syscall | 0x800034ea | 0x8000357b | syscalls.c:31 | 96 | 5.64% | 1 | 96.0 | 25 |
| memcpy | 0x80004090 | 0x80004153 | syscalls.c:433 | 91 | 5.35% | 1 | 91.0 | 28 |
| [SYS] init_tls | 0x800037b0 | 0x80003819 | syscalls.c:138 | 73 | 4.29% | 1 | 73.0 | 16 |
| [SYS] _init | 0x8000381a | 0x800038ff | syscalls.c:147 | 58 | 3.41% | 1 | 58.0 | 17 |
| printstr | 0x80003722 | 0x8000375d | syscalls.c:117 | 37 | 2.18% | 1 | 37.0 | 17 |
| [SYS] thread_entry | 0x8000375e | 0x80003785 | syscalls.c:124 | 34 | 2.00% | 1 | 34.0 | 8 |
| main | 0x80003786 | 0x800037af | syscalls.c:131 | 28 | 1.65% | 1 | 28.0 | 10 |
| [SYS] _start | 0x80000000 | 0x7fffffff | crt.S:18 | | | | | |
| [SYS] _start_text | 0x80000000 | 0x80000123 | crt.S:18 | | | | | |
| [SYS] trap_entry | 0x80000124 | 0x800001cf | crt.S:141 | | | | | |
| _exit | 0x800001d0 | 0x80002fff | crt.S:248 | | | | | |
| workload | 0x80003000 | 0x800031cb | benchmark.c:10 | | | | | |
| verify | 0x800031cc | 0x800032d9 | syscalls.c:13 | | | | | |
| verifyDouble | 0x800032da | 0x800033fd | syscalls.c:29 | | | | | |
| barrier | 0x800033fe | 0x80003487 | syscalls.c:45 | | | | | |
| lfsr | 0x80003488 | 0x800034bd | syscalls.c:67 | | | | | |

### Tab 2: System Trace

Reads: system_trace.txt
Columns: Cycle | PC | Mnemonic | Event | Function |
Reg Delta | Mem Access

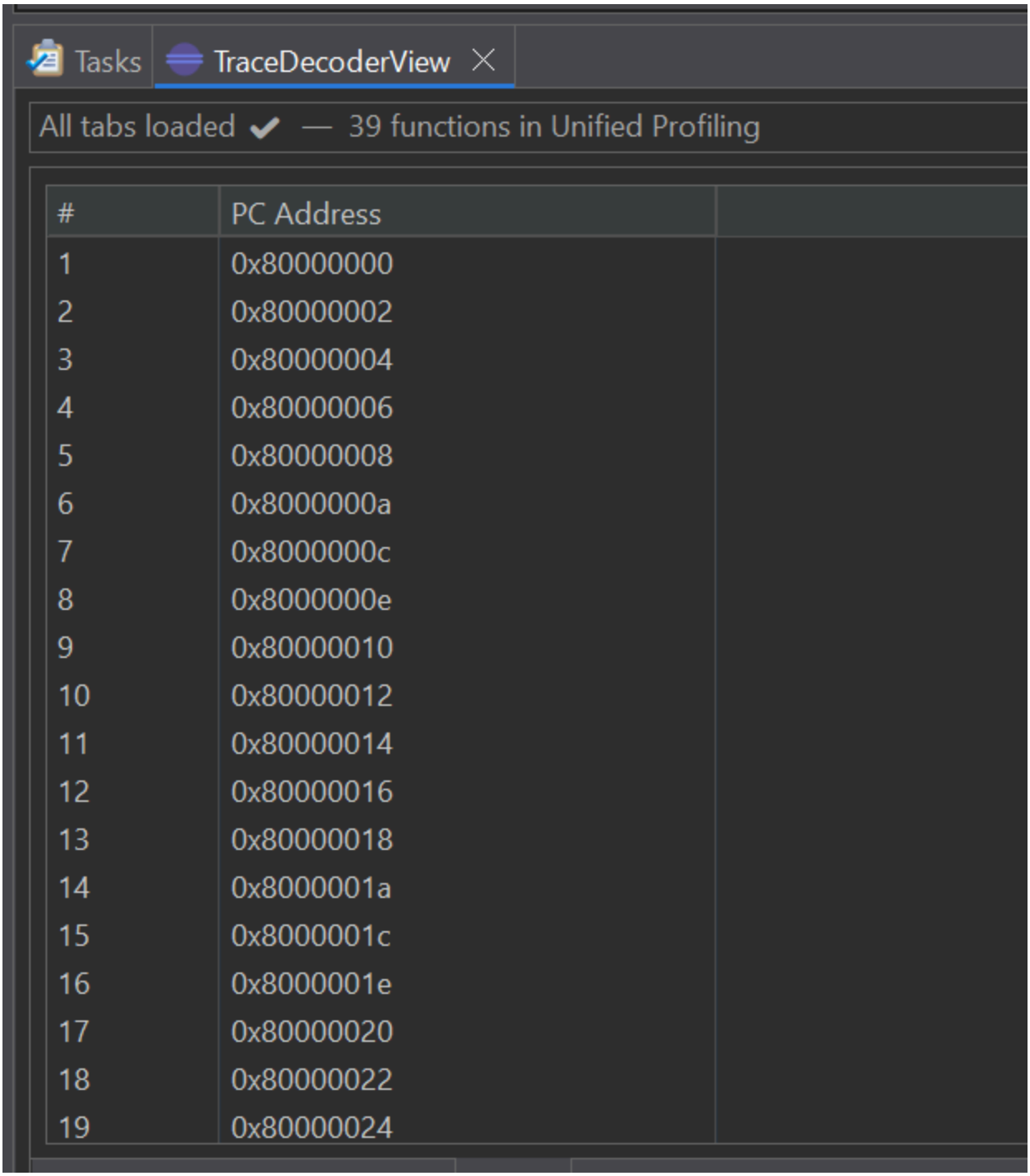
Limit: first 5,000 rows (prevents UI freeze on large traces)




| Cycle | PC | Mnemonic | Event | Function | Reg Delta | Mem Access |
|---|---|---|---|---|---|---|
| 265 | 0x0000000000010000 | addi | EXEC | ? | - | - |
| 290 | 0x0000000000010014 | jalr | RET | ? | - | - |
| 486 | 0x000000008000381a | c.arith | EXEC | _init | - | - |
| 498 | 0x0000000080003828 | sw | STORE | _init | - | W@0x80055f9c=0x00000000 |
| 498 | 0x000000008000382c | c.mv | EXEC | _init | a5=0x0000000000000001 | - |
| 507 | 0x000000008000382e | sw | STORE | _init | - | W@0x80055f98=0x00000001 |
| 508 | 0x0000000080003832 | addi | EXEC | _init | a5=0x0000000080056020 | - |
| 510 | 0x0000000080003836 | sd | STORE | _init | - | W@0x80055f90=0x80056020 |
| 526 | 0x000000008000383a | ld | LOAD | _init | a5=0x0000000080056020 | - |
| 527 | 0x000000008000383e | addi | EXEC | _init | a5=0x0000000080055fa0 | - |
| 529 | 0x0000000080003846 | c.beqz | BRANCH | _init | a5=0x0000000000200157f | - |
| 530 | 0x0000000080003848 | slli | EXEC | _init | - | - |
| 531 | 0x000000008000384c | jal | CALL | _init | - | - |
| 532 | 0x00000000800037b0 | c.arith | EXEC | init_tls | - | - |
| 544 | 0x00000000800037cc | sd | STORE | init_tls | - | W@0x80055f78=0x00000000 |
| 551 | 0x00000000800037d0 | c.mv | EXEC | init_tls | a5=0x0000000080036040 | - |
| 560 | 0x00000000800037d2 | ld | LOAD | init_tls | a2=0x0000000000000000 | - |
| 560 | 0x00000000800037d6 | auipc | EXEC | init_tls | a1=0x00000000800367d6 | - |
| 562 | 0x00000000800037e0 | jal | CALL | init_tls | - | - |

## Tab 3: E-Trace

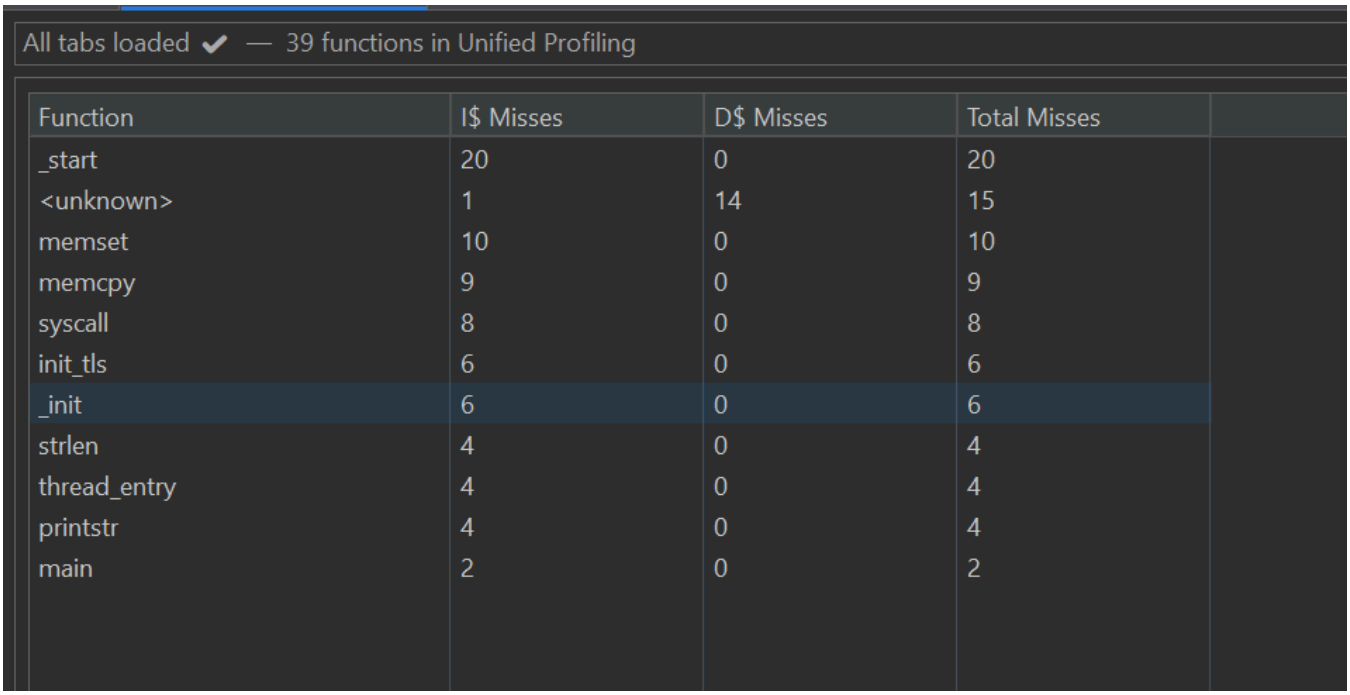
Reads: spike_pc_trace_input.decoder.trace
Columns: # (sequence number) | PC Address
Limit: first 5,000 entries




| # | PC Address |
|---|---|
| 1 | 0x80000000 |
| 2 | 0x80000002 |
| 3 | 0x80000004 |
| 4 | 0x80000006 |
| 5 | 0x80000008 |
| 6 | 0x8000000a |
| 7 | 0x8000000c |
| 8 | 0x8000000e |
| 9 | 0x80000010 |
| 10 | 0x80000012 |
| 11 | 0x80000014 |
| 12 | 0x80000016 |
| 13 | 0x80000018 |
| 14 | 0x8000001a |
| 15 | 0x8000001c |
| 16 | 0x8000001e |
| 17 | 0x80000020 |
| 18 | 0x80000022 |
| 19 | 0x80000024 |

## Tab 4: Cache Misses

Reads: rtl_profiling_report.csv
Columns: Function | I$ Misses | D$ Misses | Total Misses




A status bar above the tab folder shows the current state: "Loading pipeline results...", "Waiting for: <filename>", "New pipeline results detected… reloading...", or "All tabs loaded, N functions in Unified Profiling".




### 10.2.2 Trigger File Watcher

A daemon thread started at plugin initialization polls pipeline_done.trigger every 2 seconds:

```
private void startTriggerWatcher() {
    Thread watcher = new Thread(() -> {
        while (watcherRunning) {
            Thread.sleep(2000);
            File trigger = new File(TRIGGER_FILE);
            if (trigger.exists()) {
                long modified = trigger.lastModified();
                if (modified > lastTriggerTime) {
                    lastTriggerTime = modified;
                    Display.getDefault().asyncExec(() -> {
                        statusText.setText("New pipeline results detected — reloading...");
                        loadAllTabs();
                    });
                }
            }
        }
    });
```

```
        watcher.setDaemon(true);
        watcher.start();
    }
```

Display.getDefault().asyncExec() is used to marshal the UI update back onto the SWT event thread, which is required for all Eclipse widget modifications. The daemon flag ensures the thread does not prevent Eclipse from shutting down cleanly.

### 10.2.3 CSV Parsing: split(",", -1)

The unified profiling CSV has 20 columns (indices 0–19). The parser uses split(",", -1) with the limit parameter set to -1, which preserves trailing empty tokens. Without this, a row ending in commas (e.g., a function with no cache data) would produce fewer than 20 tokens and cause an ArrayIndexOutOfBoundsException at index 19. The -1 limit guarantees all 20 slots are always present.

Column indices are defined as named constants rather than magic numbers, so if the CSV schema changes, only the constant definitions need updating:

```
    private static final int C_FUNCTION          =  0;
    private static final int C_PC_START          =  1;
    private static final int C_TOTAL_CYCLES      =  6;
    private static final int C_ICACHE_MISSES     = 17;
    private static final int C_DCACHE_MISSES     = 18;
    private static final int C_TOTAL_CACHE_MISSES = 19;
```

A null-safe helper safeGet(String[] arr, int idx) handles any remaining edge cases by returning an empty string for out-of-bounds indices.

### 10.2.4 System Trace Parsing

The system_trace.txt file uses fixed-width columns separated by two or more spaces. The parser splits on \s{2,} and filters out separator lines (---), summary lines ([SUMMARY]), and blank lines before building table rows.

## 10.3 End-to-End Eclipse Workflow

The complete Eclipse-integrated workflow from a single terminal command:

```
export TARGET_C=/mnt/d/riscv-work/cva6/verif/tests/custom/benchmark.c
bash run_trace_pipeline1.sh
```

Step 1: Compile fibonacci.c → simulation. elf
Step 2: Spike ISA simulation → spike_raw.log → spike_pc_trace.txt
Step 3: Verilator RTL simulation → trace_hart_0.dasm + trace_rvfi_hart_00.dasm
Step 4: E-Trace encode → te_inst_raw packets → decode → verified PC trace
Step 5: System trace merge → system_trace.txt
Step 6: Function profiling → function_exec_time.csv
Step 7: Cache miss correlation → rtl_profiling_report.csv (if available)
Step 8: Unified report → unified_profiling_report.csv + .txt + .html
Step 9: Write pipeline_done.trigger
Step 10: Launch Eclipse → TraceDecoderView auto-loads all four tabs

The user opens the Trace Decoder view in Eclipse (Window → Show View → Other → Trace Decoder) and immediately sees populated tables for all four data categories. If the pipeline is re-run (e.g., after changing the target C file), the watcher detects the updated trigger file and refreshes all tables without restarting Eclipse.